

Подробное объяснение лабораторной работы

5

Флаги и условные команды. Ветвления и циклы

Команда 8. Варианты: Л5.1=2, Л5.2=2, Л5.3=2, Л5.4=2, Л5.5=2, Л5.6=2, Л5.7=2, Л5.8=2, Л5.9=3

Цель документа - дать теорию и практическое объяснение, достаточное для сдачи лабораторной работы и ответа на вопросы преподавателя.

Параметр	Значение
Язык низкого уровня	GNU Assembler, синтаксис AT&T
Связка с C++	extern "C" функции из файла lab5.S вызываются из main.cpp
Целевая среда	Qt/qmake + MinGW 64-bit, Windows x64 ABI
Главные темы	RFLAGS/EFLAGS, ADD/CMP, SETcc, CMOVcc, условные переходы, циклы, SSE сравнения, адресация массивов и матриц
Бонусы	+1 за LEA в Л5.5; +2 за Л5.7; +2 за Л5.8; +2 за Л5.9

Содержание

1. Что проверяет лабораторная работа
2. Варианты команды 8
3. Теория, которую нужно знать
4. Сборка и запуск проекта
5. Подробный разбор заданий Л5.1 - Л5.9
6. Бонусы, штрафы и как их объяснить на защите
7. Типовые вопросы на защите
8. Краткая шпаргалка
9. Источники

1. Что проверяет лабораторная работа

Лабораторная работа 5 посвящена тому, как процессор принимает решения на уровне машинных команд. В языке C++ условие вида `if (x > y)` выглядит как одна логическая конструкция, но на уровне x86 обычно раскладывается на несколько шагов: получить или сравнить значения, выставить флаги в RFLAGS/EFLAGS, затем выполнить условную команду, которая смотрит на эти флаги.

Главная практическая идея: условие не хранится в отдельной переменной. Его часто кодируют состоянием флагов: CF, OF, SF, ZF, AF, PF. Команды ADD, SUB, CMP, COMISS/COMISD изменяют флаги, а SETcc, CMOVcc, Jcc используют их.

- Л5.1 учит получать флаги после сложения и превращать выбранный флаг в 0/1.
- Л5.2 и Л5.3 показывают, что одна и та же битовая комбинация сравнивается по-разному как unsigned и signed.
- Л5.4 переносит идею сравнения на числа float через SSE/AVX.
- Л5.5 показывает ветвление без перехода: выбор результата через CMOVcc.
- Л5.6 и Л5.7 закрепляют циклы: запись последовательности в массив и печать без массива.
- Л5.8 и Л5.9 - бонусные задачи на обход массива double и плоской матрицы int32_t.

2. Варианты команды 8

Задание	Вариант	Что нужно реализовать
Л5.1	2	$z = x + y$; $w = 1$, если после ADD произошло знаковое переполнение OF.
Л5.2	2	Для unsigned x вычислить $z = (x > -3)$.
Л5.3	2	То же сравнение $z = (x > -3)$, но для signed x .
Л5.4	2	То же сравнение для float, используя SSE/AVX команды сравнения.
Л5.5	2	Для unsigned x : $z = x - 2$, если $x - 2 \geq -2$; иначе $z = 25$. Использовать CMP и CMOVcc.
Л5.6	2	Заполнить массив первыми N нечетными неотрицательными числами: 1, 3, 5, 7, ...
Л5.7	2	Напечатать первые N нечетных неотрицательных чисел без сохранения в массив.
Л5.8	2	Функция sa возвращает индекс максимального элемента массива double.
Л5.9	3	Функция mte заменяет главную диагональ квадратной плоской матрицы на -1.

3. Теория, которую нужно знать

3.1. Синтаксис GNU Assembler AT&T

В проекте используется GNU Assembler с AT&T-синтаксисом. Это важно, потому что порядок операндов отличается от Intel-синтаксиса. В AT&T порядок обычно такой: источник, приемник. Например, `movl %ecx, %eax` означает `eax = ecx`, а `addl %edx, %eax` означает `eax = eax + edx`.

Элемент	AT&T пример	Смысл
Регистр	<code>%eax</code>	Имя регистра начинается с %.
Непосредственное значение	<code>-\$3</code>	Константа начинается с \$.
Память по адресу регистра	<code>(%r8)</code>	Обращение к памяти по адресу <code>r8</code> .
Смещение	<code>8(%r8)</code>	Память по адресу <code>r8 + 8</code> .
Индексная адресация	<code>(%rcx,%r8,8)</code>	Адрес <code>rcx + r8*8</code> . Удобно для <code>double array[i]</code> .
Размер операции	<code>movl, movq, movsd</code>	Суффикс <code>l</code> - 32 бита, <code>q</code> - 64 бита, <code>sd</code> - scalar double.

На защите часто спрашивают порядок операндов `CMPL`. В AT&T запись `cmpl $-3, %ecx` выставляет флаги по результату `%ecx - (-3)`, то есть проверяет `x` относительно `-3`.

3.2. Регистры и размеры данных

В x86-64 один 64-битный регистр имеет несколько имен для младших частей. Например, `RAX` - 64 бита, `EAX` - младшие 32 бита, `AX` - младшие 16 бит, `AL` - младшие 8 бит. Запись в `EAX` в 64-битном режиме обнуляет старшие 32 бита `RAX`. Это удобно, когда функция возвращает `int`: достаточно записать результат в `EAX`.

Тип C/C++	Размер	Типичные команды/регистры
<code>int32_t / uint32_t</code>	4 байта	<code>EAX, ECX, EDX, movl, addl, cmpl</code>
<code>size_t</code> в x64	8 байт	<code>RAX, RCX, RDX, R8, movq, cmprq</code>
<code>float</code>	4 байта	XMM-регистры, <code>movd, comiss</code>
<code>double</code>	8 байт	XMM-регистры, <code>movsd, comisd</code>
<code>int32_t matrix element</code>	4 байта	Адрес элемента умножается на 4.

3.3. Флаги RFLAGS/EFLAGS

`RFLAGS` - регистр состояния процессора в x86-64. В лабораторной используются флаги результата арифметических и сравнительных команд. После `ADD`, `SUB`, `CMPL` и некоторых команд сравнения с плавающей запятой процессор выставляет отдельные биты, а условные команды интерпретируют эти биты.

Флаг	Название	Когда равен 1	Где используется
CF	Carry Flag	Был перенос из старшего бита при <code>unsigned</code> сложении или заем при <code>unsigned</code> вычитании.	Беззнаковые сравнения: <code>below/above</code> , <code>setb/seta</code> , <code>cmovae</code> .

Флаг	Название	Когда равен 1	Где используется
OF	Overflow Flag	Произошло знаковое переполнение: результат не помещается в signed диапазон.	Знаковые операции, Л5.1 вариант 2.
SF	Sign Flag	Старший бит результата равен 1.	Знаковые сравнения вместе с OF.
ZF	Zero Flag	Результат равен нулю.	Проверка равенства, циклы, JZ/JNZ, SETZ.
AF	Auxiliary Carry	Перенос/заем между битами 3 и 4.	Редко нужен; в Л5.1 его требуется вывести.
PF	Parity Flag	В младшем байте результата четное количество единичных битов.	SETPE/SETP; также участвует в unordered для float.

Пример: 2147483647 + 1 в 32-битном signed int дает битовую комбинацию 0x80000000, которая как signed равна -2147483648. Здесь unsigned-переноса нет (CF=0), но signed-переполнение есть (OF=1), потому что сложили два положительных числа, а получили отрицательное.

3.4. Дополнительный код и различие signed/unsigned

Одна и та же 32-битная последовательность может означать разные числа в зависимости от типа. Например, 0xFFFFFFFFD как uint32_t - это 4294967293, а как int32_t - это -3. Процессор не знает, "signed" перед ним или "unsigned". Это решает программист через выбор условной команды.

Биты	Как uint32_t	Как int32_t
0x00000000	0	0
0x00000001	1	1
0x7FFFFFFF	2147483647	2147483647
0x80000000	2147483648	-2147483648
0xFFFFFFFFD	4294967293	-3
0xFFFFFFFFF	4294967295	-1

Именно поэтому Л5.2 и Л5.3 дают разные ответы для одних и тех же входных битов: в Л5.2 используются unsigned-условия, в Л5.3 - signed-условия.

3.5. ADD, SUB и CMP

ADD изменяет приемник и выставляет флаги по результату сложения. CMP работает как SUB только для флагов: он вычисляет приемник - источник, выставляет флаги, но результат никуда не сохраняет. После CMP следующая команда SETcc, CMOVcc или Jcc интерпретирует флаги как отношение между операндами.

Команда	AT&T запись	Что происходит
addl	addl %edx, %eax	eax = eax + edx; флаги обновлены по сумме.
cmpl	cmpl \$-3, %ecx	Флаги как после esx - (-3); esx не меняется.
testq	testq %rdx, %rdx	Побитовое И для проверки нуля; операнд не меняется.

Команда	AT&T запись	Что происходит
decq	decq %rdx	Уменьшение счетчика на 1; часто используется в циклах.

3.6. SETсс: превращение условия в 0 или 1

SETсс записывает один байт: 1, если условие ss истинно, иначе 0. Обычно перед SETсс обнуляют EAX, затем пишут в AL. Так получается корректный int-результат 0 или 1.

```
xorl %eax, %eax
cmpl $-3, %ecx
setg %al      # eax = 1, если signed ecx > -3, иначе eax = 0
```

Команда	Условие	Интерпретация
setc / setb	CF=1	Unsigned ниже / был carry.
seta	CF=0 и ZF=0	Unsigned выше: a > b.
setae / setnb	CF=0	Unsigned выше или равно: a >= b.
setz / sete	ZF=1	Равно нулю / равно.
setnz / setne	ZF=0	Не равно.
sets	SF=1	Результат имеет старший бит 1.
seto	OF=1	Знаковое переполнение.
setg	ZF=0 и SF=OF	Signed больше.
setl	SF!=OF	Signed меньше.

3.7. CMOVсс: условное присваивание без перехода

CMOVсс копирует источник в приемник только если условие истинно. Если условие ложно, приемник остается прежним. Это полезно для ветвлений вида if/else, где нужно выбрать одно из двух значений, но не хочется делать jump.

```
leal -2(%rcx), %edx    # edx = x - 2
movl $25, %eax         # результат по умолчанию
cmpl $-2, %edx         # сравнить edx и -2 как unsigned
cmovael %edx, %eax     # если edx >= -2 unsigned, eax = edx
```

Важно: CMOVсс не выставляет флаги. Он только читает флаги, которые уже выставлены предыдущей командой, обычно CMP или TEST.

3.8. LEA как арифметическая команда

LEA означает load effective address, но ее часто используют как быструю целочисленную арифметику без изменения флагов. В Л5.5 бонус дается за вычисление линейной комбинации одной командой LEA.

```
leal -2(%rcx), %edx    # edx = ecx - 2, флаги не изменяются
```

LEA особенно удобна для выражений вида $a + b \cdot \text{scale} + \text{const}$, где scale может быть 1, 2, 4 или 8. Например, leaq (%rcx,%r8,8), %rax вычисляет адрес $\text{rcx} + \text{r8} \cdot 8$.

3.9. Условные переходы и циклы

Цикл на ассемблере обычно строится из счетчика, тела цикла и условного перехода. В этой лабораторной запрещены команды loop*, jcx*, jecx*. Вместо них используется обычная связка dec + jne или cmp + jb.

```
testq %rdx, %rdx
jz done
loop_body:
...
```

```

    decq %rdx
    jne loop_body
done:
    ret

```

Связка `dec + jne` обычно быстрее и прозрачнее, чем `loop`. Поэтому в требованиях прямо есть штраф за `loop*`, `jsx*`, `jesx*`.

3.10. Адресация массивов и матриц

Массив в C++ - это непрерывный участок памяти. Если `p` указывает на первый элемент `int32_t`, то элемент `a[i]` находится по адресу `p + i*4`. Если `p` указывает на `double`, то `a[i]` находится по адресу `p + i*8`.

Структура	Формула адреса	Пример asm
<code>int32_t a[i]</code>	<code>base + i*4</code>	<code>addq \$4, %rcx</code> после каждой записи
<code>double a[i]</code>	<code>base + i*8</code>	<code>movsd (%rcx,%r8,8), %xmm1</code>
<code>M[i][j]</code> в плоской <code>n x n</code> матрице <code>int32_t</code>	<code>base + (i*n + j)*4</code>	Для главной диагонали <code>j=i</code> : шаг <code>(n+1)*4</code>

3.11. SSE/AVX сравнение float и double

Для чисел с плавающей запятой нельзя использовать обычный `CMPL` по целым регистрам. Значения `float/double` находятся в XMM-регистрах, а сравнение выполняется командами `COMISS/COMISD` или AVX-аналогами `VCOMISS/VCOMISD`.

Команда	Тип	Что сравнивает
<code>comiss %xmm1, %xmm0</code>	float	Сравнивает нижние 32 бита <code>xmm0</code> с нижними 32 битами <code>xmm1</code> .
<code>comisd %xmm0, %xmm1</code>	double	Сравнивает нижние 64 бита <code>xmm1</code> с нижними 64 битами <code>xmm0</code> .
<code>seta</code> после <code>comiss/comisd</code>	ordered greater	Истина, если левый операнд строго больше правого и сравнение не <code>unordered</code> .

В Л5.4 используется `float`, поэтому константа `-3.0f` помещается в XMM-регистр как `0xC0400000`, затем `COMISS` выставляет флаги, а `SETA` возвращает 1 только при `x > -3`.

Не следует смешивать AVX и SSE в одном задании: за это предусмотрен штраф. В данной реализации используются SSE-команды, и AVX-команд рядом с ними нет.

3.12. Соглашение о вызовах Windows x64 ABI

Файл `lab5.S` написан под MinGW 64-bit, то есть под Windows x64 calling convention. Это критично: если тот же `asm` собрать и вызвать на Linux SysV ABI без адаптации, аргументы окажутся не в тех регистрах.

Правило Windows x64	Значение для этой лабораторной
1-й целочисленный/указательный аргумент	RCX
2-й целочисленный/указательный аргумент	RDX
3-й целочисленный/указательный аргумент	R8
4-й целочисленный/указательный аргумент	R9
float/double аргументы	XMM0-XMM3 по позиции аргумента
Возврат <code>int</code>	EAX
Возврат <code>size_t</code>	RAX

Правило Windows x64	Значение для этой лабораторной
Volatile регистры	RAX, RCX, RDX, R8-R11, XMM0-XMM5 можно портить без сохранения.
Non-volatile регистры	RBX, RBP, RSI, RDI, R12-R15, XMM6-XMM15 нужно сохранять, если функция их меняет.
Shadow space	Перед вызовом другой функции вызывающий должен иметь 32 байта домашней области на стеке.

В Л5.7 функция вызывает printf, поэтому она сохраняет RBX и RSI, выделяет место на стеке и поддерживает выравнивание перед call. Это нужно уметь объяснить, потому что нарушение ABI дает большой штраф.

3.13. Связь C++ и asm-функций

В C++ функции объявлены в блоке extern "C". Это отключает C++ name mangling, поэтому имена lab5_1, lab5_2 и т.д. совпадают с глобальными метками в asm-файле. Без extern "C" компоновщик искал бы имена с C++-декорацией и не нашел бы функции.

```
extern "C" {
    void lab5_1(std::int32_t x, std::int32_t y, Lab5_1_Result *result);
    std::int32_t lab5_2(std::uint32_t x);
    ...
}
```

В реализации нет inline assembly, поэтому нет clobber-списков. Это снимает риск штрафа за неправильные clobbers во вставках. С другой стороны, вся ответственность за ABI переносится на asm-функции.

4. Сборка и запуск проекта

Проект рассчитан на Qt Creator с qmake и комплектом Desktop Qt MinGW 64-bit. Файл lab5_team8_ready.pro добавляет lab5.S в сборку как assembler-with-cpp. После сборки запускается консольное приложение, в котором можно выбрать номер задания от 1 до 9.

Lab 5, team number 8, Qt 6.7.3/qmake + Desktop Qt MinGW 64-bit.

Open lab5_team8_ready.pro in Qt Creator and build as a console application.
The assembly source is lab5.S. It is compiled by qmake through gcc
-x assembler-with-cpp, so it is compatible with MinGW/GNU assembler.

Implemented variants:

Lab5.1 = 2: $z = x + y$; $w = 1$ iff signed overflow OF occurred.
Lab5.2 = 2: unsigned $z = (x > -3)$, C/C++ unsigned comparison semantics.
Lab5.3 = 2: signed $z = (x > -3)$.
Lab5.4 = 2: float $z = (x > -3)$, SSE comiss only; no AVX/SSE mixing.
Lab5.5 = 2: cmovcc + cmp; $x - 2$ is computed by one lea for the bonus.
Lab5.6 = 2: first N odd non-negative numbers are written to an array.
Lab5.7 = 2: first N odd non-negative numbers are printed without an array;
implemented as a complete assembly function.
Lab5.8 = 2: sa returns the 0-based index of the maximum double element.
Lab5.9 = 3: mre replaces the main diagonal of a flat int matrix by -1.

Penalty-related choices:

- no inline assembly clobber lists are used;
- no local variables are stored in the data segment;
- no loop*, jcx*, or jecx* instructions are used;
- matrix storage is a contiguous vector<int32_t>, not an array of row pointers;

- all non-volatile registers modified by the assembly printf loop are saved/restored;
- all dynamic memory is managed by std::vector.
- Открыть lab5_team8_ready.pro в Qt Creator.
- Выбрать комплект Desktop Qt MinGW 64-bit.
- Собрать проект как консольное приложение.
- Запустить, выбрать номер задания, ввести данные, проверить результат.

Если преподаватель собирает на Linux GCC, нужно переписать передачу аргументов под SysV ABI: первый аргумент будет в RDI, второй в RSI, третий в RDX и т.д. Текущий код корректен именно для MinGW/Windows x64.

5. Подробный разбор заданий Л5.1 - Л5.9

5.1. Л5.1 - сложение, флаги и SETcc

Формулировка для команды 8

Вычислить $z = x + y$ командой ADD. Для варианта 2 сформировать $w = 1$, если после сложения произошло знаковое переполнение OF; иначе $w = 0$. Дополнительно вывести CF, OF, SF, ZF, AF, PF.

Что нужно понимать

- ADD не только считает сумму, но и выставляет флаги результата.
- OF отвечает за signed overflow; CF отвечает за unsigned carry. Это разные вещи.
- SETcc записывает 0 или 1 в байт памяти/регистр в зависимости от конкретного флага или условия.
- Для AF нет удобной SETcc-команды, поэтому флаг извлекается из RFLAGS через pushfq/popq и сдвиг.

Алгоритм реализации

1. Скопировать x из ECX в EAX.
2. Выполнить `addl %edx, %eax`, то есть $z = x + y$.
3. Сохранить z по адресу структуры result.
4. Командами `setc/seto/sets/setz/setp` записать флаги CF, OF, SF, ZF, PF.
5. Для w еще раз использовать `seto`, расширить AL до EAX и записать `int w`.
6. Для AF сохранить RFLAGS в RAX, сдвинуть бит 4 в младший бит и записать его.

Ключевой фрагмент кода

```
lab5_1:
    movl %ecx, %eax
    addl %edx, %eax
    movl %eax, (%r8)
    setc 8(%r8)
    seto 9(%r8)
    sets 10(%r8)
    setz 11(%r8)
    setp 13(%r8)
    seto %al
    movzbl %al, %eax
    movl %eax, 4(%r8)
    pushfq
    popq %rax
    shrq $4, %rax
    andb $1, %al
    movb %al, 12(%r8)
    ret
```

Контрольный запуск / ожидаемый результат

```
Task = 1
x = 2147483647
y = 1
z = x + y = -2147483648
w = 1 (1, если было знаковое переполнение OF)
CF=0 OF=1 SF=1 ZF=0 AF=1 PF=1
```

Как объяснить на защите

- Команда `addl` работает с 32-битными значениями, поэтому результат хранится в `EAX` и интерпретируется как `int32_t`.
- При $2147483647 + 1$ беззнакового переноса нет, но знаковое переполнение есть: два положительных операнда дали отрицательный `signed`-результат.
- `w` зависит именно от `OF`, потому что вариант 2 спрашивает знаковое переполнение.
- `AF` извлекается отдельно, потому что `SETcc` не имеет команды `"setaf"`.

Типичные ошибки

- Путать `CF` и `OF`. `CF` нужен для `unsigned`, `OF` - для `signed`.
- Записать только `w` и забыть вывести все требуемые флаги.
- Извлечь `AF` неверным битом: `AF` - это бит 4 регистра `FLAGS`.

5.2. Л5.2 - unsigned сравнение $x > -3$

Формулировка для команды 8

Для целого беззнакового `x` вычислить $z = (x > -3)$. Результат должен быть 0 или 1, как у операторов сравнения `C/C++`.

Что нужно понимать

- В `unsigned`-выражении константа `-3` преобразуется к `uint32_t`: `0xFFFFFFFFD = 4294967293`.
- Условие $x > -3$ для `uint32_t` истинно только для $x = 4294967294$ и $x = 4294967295$.
- Для `unsigned` "больше" используется `SETA`, а не `SETG`.
- `CMPL` только выставляет флаги, значение `ECX` не меняет.

Алгоритм реализации

7. Обнулить `EAX`, чтобы весь `int`-результат был 0.
8. Сравнить `ECX` с `-3`: `cmpl $-3, %ecx`.
9. Выполнить `seta %al`. `SETA` означает `unsigned above`: `CF=0` и `ZF=0`.
10. Вернуть `EAX`.

Ключевой фрагмент кода

```
lab5_2:
    xorl %eax, %eax
    cmpl $-3, %ecx
    seta %al
    ret
```

Контрольный запуск / ожидаемый результат

```
Task = 2
unsigned x = 10
z = (x > -3) = 0
```

```
Task = 2
```

```
unsigned x = 4294967294
z = (x > -3) = 1
```

Как объяснить на защите

- Нужно проговорить, что -3 в unsigned сравнении - не отрицательное число, а 4294967293.
- SETA выбран потому, что задание требует беззнаковое сравнение.
- Если заменить SETA на SETG, получится другое условие и ошибка.

Типичные ошибки

- Считать, что unsigned x > -3 всегда истинно. В C/C++ сначала происходит преобразование типа.
- Использовать signed-команду setg вместо unsigned-команды seta.

5.3. Л5.3 - signed сравнение x > -3

Формулировка для команды 8

Реализовать то же выражение $z = (x > -3)$, но теперь x является знаковым целым.

Что нужно понимать

- Для signed int значение -3 действительно является отрицательным числом -3.
- Знаковое сравнение использует SF и OF, поэтому команда должна быть SETG.
- SETG истинна, когда ZF=0 и SF=OF.

Алгоритм реализации

11. Обнулить EAX.
12. Сравнить ECX с -3.
13. Выполнить setg %al для signed greater.
14. Вернуть 0 или 1 в EAX.

Ключевой фрагмент кода

```
lab5_3:
    xorl %eax, %eax
    cmpl $-3, %ecx
    setg %al
    ret
```

Контрольный запуск / ожидаемый результат

```
Task = 3
signed x = -3
z = (x > -3) = 0
```

```
Task = 3
signed x = -2
z = (x > -3) = 1
```

Как объяснить на защите

- Эта задача отличается от Л5.2 только выбором интерпретации флагов.
- CMP тот же, но SETcc другой: setg для signed, seta для unsigned.
- На уровне битов процессор не знает тип; тип реализуется выбором условной команды.

Типичные ошибки

- Использовать setae/seta по привычке из Л5.2.
- Забыть, что equality не проходит: $-3 > -3$ ложно, поэтому setg, а не setge.

5.4. Л5.4 - сравнение float через SSE COMISS

Формулировка для команды 8

Реализовать $z = (x > -3)$ для x типа float. Вариант 2 таблицы типов - одинарная точность float. Используются SSE-аналоги AVX-команд сравнения.

Что нужно понимать

- float хранится и передается через XMM-регистр, а не через ECX.
- Константа -3.0f имеет IEEE-754 битовое представление 0xC0400000.
- COMISS сравнивает scalar single precision значения и выставляет CF/ZF/PF.
- SETA после COMISS соответствует ordered greater: $x > -3$ и сравнение не unordered.
- В одном задании нельзя смешивать AVX и SSE; здесь используются только SSE-команды.

Алгоритм реализации

15. Записать битовую константу -3.0f в EAX.
16. Перенести ее в XMM1 через MOVD.
17. Обнулить EAX для результата 0 по умолчанию.
18. Сравнить XMM0 с XMM1 командой COMISS.
19. Командой SETA записать 1, если x строго больше -3.0f.

Ключевой фрагмент кода

```
lab5_4:
    movl $0xC0400000, %eax
    movd %eax, %xmm1
    xorl %eax, %eax
    comiss %xmm1, %xmm0
    seta %al
    ret
```

Контрольный запуск / ожидаемый результат

```
Task = 4
float x = -3
z = (x > -3) = 0
```

```
Task = 4
float x = -2.5
z = (x > -3) = 1
```

Как объяснить на защите

- XMM0 содержит первый float-аргумент по Windows x64 ABI.
- COMISS %xmm1, %xmm0 сравнивает xmm0 с xmm1, то есть x с -3.0f.
- SETA возвращает 1 только для строгого "больше".
- SSE используется последовательно, без AVX-инструкций, поэтому штрафа за смешение нет.

Типичные ошибки

- Сравнивать float обычным cmpl по битам - это неверная числовая семантика.
- Использовать double-команду COMISD вместо float-команды COMISS.
- Смешать vcomiss и movss/comiss в одном решении без необходимости.

5.5. Л5.5 - CMOVcc и LEA

Формулировка для команды 8

Для unsigned x реализовать вариант 2: $z = x - 2$, если $x - 2 \geq -2$; иначе $z = 25$. Нужно использовать CMP для флагов и CMOVcc для выбора результата. Бонус: линейная комбинация $x - 2$ вычисляется одной командой LEA.

Что нужно понимать

- x имеет тип unsigned, поэтому выражение и сравнение рассматриваются в арифметике modulo 2^{32} .
- Константа -2 как uint32_t равна $0xFFFFFFFF$ = 4294967294.
- CMOVcc позволяет заменить if/else без условного перехода.
- LEA вычисляет $x - 2$ без изменения флагов. Это важно: флаги выставляет именно CMP.

Алгоритм реализации

20. Вычислить $edx = x - 2$ одной командой LEA.
21. Положить в EAX значение по умолчанию 25.
22. Сравнить EDX с -2 как unsigned: `cmpl $-2, %edx`.
23. Если $EDX \geq -2$ unsigned, выполнить `cmovae` и заменить EAX на EDX.
24. Вернуть EAX.

Ключевой фрагмент кода

```
lab5_5:
    leal -2(%rcx), %edx
    movl $25, %eax
    cmpl $-2, %edx
    cmovael %edx, %eax
    ret
```

Контрольный запуск / ожидаемый результат

```
Task = 5
unsigned x = 0
z = 4294967294 (signed view: -2)
```

```
Task = 5
unsigned x = 1
z = 4294967295 (signed view: -1)
```

```
Task = 5
unsigned x = 2
z = 25 (signed view: 25)
```

Как объяснить на защите

- Из-за unsigned арифметики $x=0$ дает $x-2 = 0xFFFFFFFF$, и это равно -2 в unsigned-представлении; условие истинно.
- $x=1$ дает $0xFFFFFFFF$, это unsigned больше $0xFFFFFFFF$; условие истинно.
- $x=2$ дает 0, а 0 меньше $0xFFFFFFFF$ в unsigned сравнении; поэтому возвращается 25.
- CMOVAE выбран для unsigned $\geq$, потому что условие "above or equal" означает $CF=0$.

Типичные ошибки

- Интерпретировать условие как signed и ожидать обычную математику $x-2 \geq -2$ для всех x.
- Использовать CMOVGE вместо CMOVAE.
- Вычислить $x-2$ через `sub` перед `cmp` и случайно потерять нужные флаги или бонус за LEA.

5.6. Л5.6 - заполнение массива циклом

Формулировка для команды 8

Заполнить массив из N целочисленных элементов первыми N нечетными неотрицательными числами: 1, 3, 5, 7, 9, ...

Что нужно понимать

- Массив `int32_t` хранится подряд; следующий элемент находится через 4 байта.
- Первое нечетное неотрицательное число - 1, каждое следующее получается добавлением 2.
- Для цикла можно использовать счетчик N и переход JNE после DEC.
- Нужно корректно обработать N=0, чтобы не записывать в пустой массив.

Алгоритм реализации

25. Проверить RDX (N). Если N=0, сразу выйти.
26. Установить EAX=1 - текущее значение последовательности.
27. Записать EAX по адресу RCX.
28. Увеличить EAX на 2.
29. Сдвинуть указатель RCX на 4 байта.
30. Уменьшить счетчик RDX и повторять, пока он не равен нулю.

Ключевой фрагмент кода

```
lab5_6:
    testq %rdx, %rdx
    jz lab5_6_done
    movl $1, %eax
lab5_6_next:
    movl %eax, (%rcx)
    addl $2, %eax
    addq $4, %rcx
    decq %rdx
    jne lab5_6_next
lab5_6_done:
    ret
```

Контрольный запуск / ожидаемый результат

```
Task = 6
N = 5
1 3 5 7 9
```

Как объяснить на защите

- RCX - адрес начала массива, RDX - длина N по Windows x64 ABI.
- Адрес увеличивается на 4, потому что `int32_t` занимает 4 байта.
- Команды `loop/jcx/jecx` не используются, поэтому штрафа за них нет.

Типичные ошибки

- Не проверить N=0 и выполнить лишнюю запись.
- Увеличивать указатель на 1 вместо 4.
- Использовать массив указателей там, где нужен обычный непрерывный массив.

5.7. Л5.7 - печать последовательности без массива

Формулировка для команды 8

Бонусное задание: напечатать первые N нечетных неотрицательных чисел, не сохраняя их в массиве. Так как вызов функций из inline asm является неопределенным поведением, решение должно быть целой asm-функцией.

Что нужно понимать

- Функция сама вызывает printf, значит должна соблюдать Windows x64 ABI.
- RBX и RSI являются non-volatile регистрами, поэтому при изменении их надо сохранить и восстановить.
- Перед call printf нужен shadow space и корректное выравнивание стека.
- Последовательность такая же, как в Л5.6: 1, 3, 5, ...

Алгоритм реализации

31. Сохранить RBX и RSI на стеке.
32. Выделить 40 байт: 32 байта shadow space и выравнивание.
33. Поместить N в RBX, текущее значение 1 в ESI.
34. В цикле вызвать printf("%d ", value).
35. Увеличить value на 2 и уменьшить счетчик.
36. В конце напечатать перевод строки, восстановить стек и регистры.

Ключевой фрагмент кода

```
lab5_7:
    pushq %rbx
    pushq %rsi
    subq $40, %rsp
    movq %rcx, %rbx
    movl $1, %esi
    testq %rbx, %rbx
    jz lab5_7_newline
lab5_7_next:
    leaq fmt_int(%rip), %rcx
    movl %esi, %edx
    xorl %eax, %eax
    call printf
    addl $2, %esi
    decq %rbx
    jne lab5_7_next
lab5_7_newline:
    leaq fmt_newline(%rip), %rcx
    xorl %eax, %eax
    call printf
    addq $40, %rsp
    popq %rsi
    popq %rbx
    ret
```

Контрольный запуск / ожидаемый результат

```
Task = 7
N = 5
1 3 5 7 9
```

Как объяснить на защите

- Это не ассемблерная вставка, а полноценная функция `lab5_7`, поэтому вызов `printf` допустим.
- Сохранение `RBX/RSI` нужно, потому что вызывающая C++ функция имеет право ожидать, что эти регистры не изменятся.
- Строки формата лежат в `.rdata`; это константы, а не локальные переменные, поэтому штрафа за `data`-сегмент нет.

Типичные ошибки

- Вызвать `printf` из `inline asm`.
- Не выделить `shadow space` перед `call`.
- Изменить `RBX/RSI` и не восстановить их.

5.8. Л5.8 - поиск максимального double

Формулировка для команды 8

Бонусное задание: разработать функцию `size_t sa(void *p, size_t N, double x)`, которая для варианта 2 возвращает индекс максимального элемента массива `double`. Параметр `x` в этом варианте не используется.

Что нужно понимать

- `double` занимает 8 байт, поэтому адрес `a[i]` равен `p + i*8`.
- Для сравнения `double` используется `COMISD`.
- Индекс возвращается как `size_t` в `RAX`.
- Для пустого массива в реализации возвращается `SIZE_MAX`, чтобы C++ мог вывести "array is empty".

Алгоритм реализации

37. Если `N=0`, вернуть `-1` как `size_t`, то есть `SIZE_MAX`.
38. Взять `a[0]` как текущий максимум, индекс `0`.
39. Начать цикл с `i=1`.
40. Загрузить `a[i]` в `XMM1`.
41. Сравнить `candidate` с `current max` командой `COMISD`.
42. Если `candidate <= max`, не обновлять. Иначе обновить максимум и индекс.
43. Вернуть индекс максимума.

Ключевой фрагмент кода

```
sa:
    testq %rdx, %rdx
    jnz sa_nonempty
    movq $-1, %rax
    ret
sa_nonempty:
    movsd (%rcx), %xmm0
    xorl %eax, %eax
    movl $1, %r8d
    cmpq $1, %rdx
    jbe sa_done
sa_next:
    movsd (%rcx,%r8,8), %xmm1
    comisd %xmm0, %xmm1
    jbe sa_no_update
    movsd %xmm1, %xmm0
    movq %r8, %rax
sa_no_update:
```

```

    incq %r8
    cmpq %rdx, %r8
    jnb sa_next
sa_done:
    ret

```

Контрольный запуск / ожидаемый результат

```

Task = 8
N = 4
a[0] = 1.5
a[1] = -10
a[2] = 7.25
a[3] = 7
index = 2
max = 7.25

```

Как объяснить на защите

- RCX - адрес массива, RDX - N. Это Windows x64 ABI.
- XMM0 хранит текущий максимум, RAX - индекс текущего максимума, R8 - текущий индекс цикла.
- COMISD %xmm0, %xmm1 сравнивает XMM1 с XMM0; если XMM1 больше, обновляем максимум.
- При равных значениях индекс не обновляется, то есть остается первый максимум.

Типичные ошибки

- Использовать целочисленный CMP для double.
- Умножать индекс на 4 вместо 8.
- Не обработать пустой массив.

5.9. Л5.9 - замена главной диагонали матрицы

Формулировка для команды 8

Бонусное задание: для варианта 3 функция `mre(void *pM, size_t N)` заменяет главную диагональ квадратной матрицы $N \times N$ на -1. Матрица хранится плоским непрерывным блоком `int32_t`, а не массивом указателей.

Что нужно понимать

- Плоская матрица $M[i][j]$ хранится как $M[i*N + j]$.
- Главная диагональ - элементы $M[0][0]$, $M[1][1]$, ..., $M[N-1][N-1]$.
- Индекс в плоском массиве для диагонали: $i*N + i = i*(N+1)$.
- Так как `int32_t` занимает 4 байта, шаг между диагональными элементами равен $(N+1)*4$ байт.

Алгоритм реализации

44. Если $N=0$, выйти.
45. Вычислить шаг: $R8 = (N+1)*4$.
46. Записать -1 по текущему адресу.
47. Перейти к следующему диагональному элементу, добавив шаг к указателю.
48. Повторить N раз.

Ключевой фрагмент кода

```

mre:
    testq %rdx, %rdx
    jz mre_done
    leaq 1(%rdx), %r8
    shlq $2, %r8

```

```

mre_next:
    movl $-1, (%rcx)
    addq %r8, %rcx
    decq %rdx
    jne mre_next
mre_done:
    ret

```

Контрольный запуск / ожидаемый результат

```

Task = 9
N = 3
M[0][0] = 1
M[0][1] = 2
M[0][2] = 3
M[1][0] = 4
M[1][1] = 5
M[1][2] = 6
M[2][0] = 7
M[2][1] = 8
M[2][2] = 9
Before:
    1   2   3
    4   5   6
    7   8   9
After:
   -1   2   3
    4  -1   6
    7   8  -1

```

Как объяснить на защите

- Решение не использует вложенные ветвления для расчета i и j ; адреса считаются арифметикой.
- Матрица - обычный `vector<int32_t>`, то есть непрерывный блок памяти, не массив указателей.
- Шаг $(N+1)*4$ сразу переводит указатель с $M[i][i]$ на $M[i+1][i+1]$.

Типичные ошибки

- Сделать `matrix` as `int**` или `vector<vector<int>>`, что нарушает требование о плоской матрице.
- Использовать вложенные `if` внутри цикла по всем элементам вместо прохода только по диагонали.
- Забыть умножение на 4 байта для `int32_t`.

6. Бонусы, штрафы и как их объяснить на защите

В лабораторной важно не только получить правильный результат, но и не получить штрафы за технические нарушения. Ниже - практический чек-лист именно для этой реализации.

Пункт	Требование/штраф	Состояние реализации
Л5.5 LEA	+1 балл, если линейная комбинация одной LEA	Выполнено: <code>leal -2(%rcx), %edx</code> .
Л5.7	+2 балла	Выполнено: печать без массива в полноценной <code>asm</code> -функции.
Л5.8	+2 балла	Выполнено: <code>sa</code> ищет максимум <code>double</code> и возвращает индекс.
Л5.9	+2 балла	Выполнено: <code>mre</code> меняет главную диагональ плоской матрицы.
Clobbers	-1 за некорректные clobbers во вставках	Inline <code>asm</code> не используется, <code>clobber</code> -списков нет.

Пункт	Требование/штраф	Состояние реализации
AVX/SSE	-2 за смешение AVX и SSE в задании	В Л5.4 и Л5.8 используются SSE-команды; AVX не смешивается.
Data segment	-2 за локальные переменные в сегменте данных	Локальные переменные в .data/.rdata не хранятся; в .rdata только форматные строки printf.
ABI	-3 за нарушение соглашения о вызовах	Под Windows x64 ABI аргументы, return и non-volatile регистры соблюдены.
Memory leak	-3 за утечку памяти	C++ использует std::vector, ручного new/delete нет.
loop*/jcx*	-3 за loop*, jcx*, jecx*	Не используются; циклы построены через dec/jne или cmp/jb.
Матрица	-2 за массив указателей вместо матрицы	Используется плоский vector<int32_t>.

7. Типовые вопросы на защите

Вопрос: Почему в Л5.2 SETA, а в Л5.3 SETG?

Ответ: Потому что Л5.2 - unsigned сравнение, где "больше" определяется флагами CF и ZF. SETA истинна при CF=0 и ZF=0. Л5.3 - signed сравнение, где нужны SF, OF и ZF; SETG истинна при ZF=0 и SF=OF.

Вопрос: Чем CF отличается от OF?

Ответ: CF показывает unsigned перенос/заем за пределы разрядности. OF показывает signed переполнение, когда математический результат не помещается в signed диапазон.

Вопрос: Почему -3 в unsigned сравнении превращается в большое число?

Ответ: В C/C++ при сравнении с unsigned отрицательная константа приводится к unsigned по модулю 2^{32} . Поэтому -3 становится 0xFFFFFFFF, то есть 4294967293.

Вопрос: Что делает CMP?

Ответ: CMP выполняет вычитание только для флагов. В AT&T cmpl src, dst выставляет флаги как после dst - src, но dst не изменяет.

Вопрос: Почему AF извлечен через pushfq/popq?

Ответ: Для AF нет прямого условия SETcc. AF - бит 4 регистра FLAGS, поэтому сохраняем RFLAGS в RAX, сдвигаем на 4 и берем младший бит.

Вопрос: Почему LEA считается бонусом в Л5.5?

Ответ: Потому что линейная комбинация x - 2 вычислена одной командой leal -2(%rcx), %edx. LEA не меняет флаги, а флаги затем выставляет CMP.

Вопрос: Почему в Л5.4 используется COMISS?

Ответ: Потому что вариант требует float, то есть scalar single precision. Для double была бы COMISD.

Вопрос: Почему нельзя смешивать AVX и SSE?

Ответ: Это ухудшает переносимость и может давать penalty по производительности из-за переходов между состояниями AVX/SSE. В условиях лабораторной за это отдельный штраф.

Вопрос: Почему Л5.7 должна быть asm-функцией, а не inline-вставкой?

Ответ: Вызов функций из ассемблерной вставки является неопределенным поведением. Поэтому печать через printf сделана в полноценной функции lab5_7.

Вопрос: Зачем сохранять RBX и RSI в Л5.7?

Ответ: По Windows x64 ABI это non-volatile регистры. Если asm-функция их меняет, она обязана восстановить исходные значения перед ret.

Вопрос: Как вычисляется адрес главной диагонали в Л5.9?

Ответ: В плоской матрице $M[i][j] = \text{base} + (i*N+j)*4$. Для главной диагонали $j=i$, значит $\text{base} + i*(N+1)*4$. Поэтому шаг равен $(N+1)*4$.

Вопрос: Почему в Л5.8 индекс double умножается на 8?

Ответ: double занимает 8 байт, поэтому $a[i]$ находится по адресу $\text{base} + i*8$.

8. Краткая шпаргалка

8.1. Условия для целочисленных сравнений

Смысл	Unsigned команда	Signed команда
$a == b$	sete / setz	sete / setz
$a != b$	setne / setnz	setne / setnz
$a > b$	seta	setg
$a \geq b$	setae	setge
$a < b$	setb	setl
$a \leq b$	setbe	setle

8.2. Что помнить про AT&T

- Порядок операндов: источник, приемник.
- Константа: \$число.
- Регистр: %rax, %eax, %al и т.д.
- Память: (%rcx), 8(%r8), (%rcx,%r8,8).
- CMP: `cmpl src, dst` выставляет флаги для `dst - src`.
- Суффиксы размеров: b=8 бит, w=16 бит, l=32 бита, q=64 бита.

8.3. Мини-чек-лист перед сдачей

- Уметь объяснить, почему Л5.2 и Л5.3 разные.
- Уметь руками разобрать пример signed overflow для Л5.1.
- Уметь назвать флаги CF, OF, SF, ZF, AF, PF.

- Уметь объяснить, почему Л5.5 для $x=0$ возвращает 0xFFFFFFFFE.
- Уметь объяснить Windows x64 ABI: RCX/RDX/R8/R9, XMM0, return in EAX/RAX, non-volatile регистры.
- Уметь вывести формулу адреса главной диагонали: $\text{base} + i \cdot (N+1) \cdot 4$.
- Уметь показать, что бонусные задания выполнены и штрафные ситуации отсутствуют.

9. Источники

- Лабораторная работа 5: "Флаги и условные команды. Ветвления и циклы".
- Учебное пособие "Организация ЭВМ и Ассемблер", приложение с лабораторным практикумом.
- Исходники проекта: lab5.S, main.cpp, README.txt из архива lab5_team8_ready.zip.
- Соглашение о вызовах Windows x64 и документация по GNU Assembler использованы в части, соответствующей реализации MinGW 64-bit.